Building a Chatbot with Memory in LangGraph

From a single-turn LangGraph workflow to a true conversational chatbot — state design, the stateless-invoke trap, and solving it with checkpointer-based persistence.

WHAT THIS CHAPTER COVERS

- Chatbot as a single-node sequential workflow
- ChatState design with BaseMessage & add_messages
- Building the chat_node and compiling the graph
- The console chat loop pattern
- Why repeated invoke() calls forget everything
- Persistence: checkpointers and RAM vs. database storage
- MemorySaver, thread_id, and the config dictionary
- Inspecting saved state with get_state()

Building a Chatbot with Memory in LangGraph

RECAP — CHAPTERS 1–8

Earlier chapters covered LangGraph fundamentals and Agentic AI basics, then four workflow patterns built with `StateGraph`: **sequential** (single/multi-node pipelines), **parallel** (fan-out/fan-in with reducers), **conditional** (branching via routing functions), and **iterative/looping** (Generate → Evaluate → Optimize with a loop-closing edge). This chapter starts a new multi-part build: a single chatbot project that will progressively gain RAG, tools, a UI, LangSmith tracing, memory, persistence, human-in-the-loop, and fault tolerance. This part builds the chatbot's basic skeleton and exposes why it needs *persistence* to actually remember a conversation.

1. The Chatbot as a LangGraph Workflow

Definition

A chatbot built with LangGraph is not a special construct — it is simply an *LLM-based workflow*, structurally simpler than most workflows seen so far. It is a **sequential workflow with exactly one node**: a user message enters at `START`, an LLM inside a single node generates a reply, and the reply exits at `END`. That single request/response cycle then runs repeatedly, once per turn, for as long as the user keeps chatting.

Why It Exists

Every chat product — from a simple Q&A assistant to a full agent — is built on this same one-node exchange at its core. Understanding it cleanly, including its state design, is the foundation this chapter's multi-video chatbot project builds on: RAG, tools, memory, persistence, and human-in-the-loop will all be layered onto this same basic skeleton in later chapters.

How It Works

1. The workflow starts at `START` and passes the user's message to a single node, conventionally named **chat_node**.
2. Inside `chat_node`, an LLM receives the message and generates a reply.
3. `chat_node` connects directly to `END` — the reply is returned and the run finishes.
4. This entire cycle is repeated in a loop, driven by the surrounding Python code (not by the graph itself), once per user turn, until the user exits.

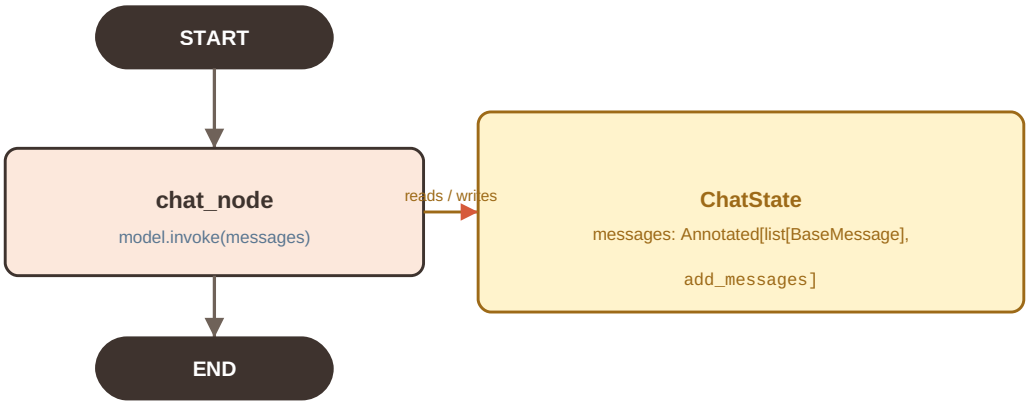

Figure 9.1 — The chatbot is a single-node sequential workflow: START feeds the user message to chat_node, which invokes the LLM and hands the reply to END.

State Design

Before writing any node logic, every LangGraph workflow needs its **state** defined first — the shared data structure every node reads from and writes to. For a chatbot, the only data that matters is the **full history of exchanged messages** between the user and the LLM. So the state holds one attribute, `messages`, containing every message exchanged so far.

1. WHY BASEMESSAGE, NOT STR

A first instinct might be to type `messages` as a list of plain strings. A better design uses **`BaseMessage`** instead. Any time content is exchanged with an LLM, it is expressed in terms of typed messages: **HumanMessage** (what the user sends), **AIMessage** (what the LLM generates), **SystemMessage** (used to set the LLM's role/persona), and **ToolMessage** (tool call results). All four types inherit from `BaseMessage`, so declaring the list as `list[BaseMessage]` gives the flexibility to hold any mix of these message types in one list, rather than being restricted to raw text.

2. WHY A REDUCER IS REQUIRED

LangGraph state has a default behavior: writing a new value to a state key **replaces** the previous value rather than adding to it. For a chatbot this is exactly wrong — every new AI reply would erase the previous conversation instead of extending it. The fix is the same one used for parallel workflows: attach a **reducer function** via `Annotated` so that new messages are appended instead of replacing what's already there.

```
from langchain_core.messages import HumanMessage, BaseMessage
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph.message import add_messages
from typing import TypedDict, Annotated
from langchain_groq import ChatGroq
from dotenv import load_dotenv

# ChatState: the only thing a chatbot needs to track is message history
class ChatState(TypedDict):
    messages : Annotated[list[BaseMessage], add_messages]
```

Instead of the generic `operator.add` reducer used in earlier chapters, this state uses **`add_messages`**, imported from `langgraph.graph.message`. Functionally it behaves like `operator.add` (it appends rather than replaces) but it is purpose-built and optimized to work with `BaseMessage` objects — which is why LangGraph recommends `add_messages` specifically whenever a state field is a list of messages.

> **IMPORTANT**
>
> `add_messages` is not just a renamed `operator.add`. It is the LangGraph-native reducer for message lists and is the recommended choice any time a state attribute holds `BaseMessage` objects, rather than manually reaching for `operator.add`.

> ↻ **Quick Revision — Chatbot as a Workflow**
>
> • A LangGraph chatbot is a sequential workflow with a single node (chat_node) between START and END.
>
> • State holds one attribute: messages, typed as Annotated[list[BaseMessage], add_messages].
>
> • BaseMessage is the common parent of HumanMessage, AIMessage, SystemMessage, and ToolMessage.
>
> • A reducer is required because LangGraph state normally replaces values rather than accumulating them.
>
> • add_messages is the LangGraph-native, BaseMessage-optimized equivalent of operator.add.

2. Implementing the Chatbot

The Chat Node

With the state defined, the node function is straightforward: pull the current messages out of state, send them to the LLM, and return the new AI reply so it can be merged back into state by the reducer.

```
model = ChatGroq(model = "openai/gpt-oss-20b")

def chat_node(state : ChatState):

    # Fetch user query
    messages = state['messages']

    # Send query to LLM
    response = model.invoke(messages)

    # Save response
    return {'messages':[response]}
```

Three points are worth calling out:

1. **The full message list is sent, not just the latest one.** `model.invoke(messages)` passes the entire `messages` list from state, so the LLM sees the whole conversation so far (whatever is currently in state) — not only the newest message.

2. **The return value is wrapped in a list.** `{'messages': [response]}` returns a single-item list because the reducer expects a list to merge; `add_messages` takes that one new AI message and appends it onto the existing list already sitting in state.
3. **The node does not need to know about history management itself.** Appending is entirely delegated to the reducer declared on the state schema — the node just returns what's new.

Assembling the Graph

Graph assembly follows the same five-step pattern used throughout this project: create the graph with the state schema, add the node, wire the edges, compile, and store the result.

```
graph = StateGraph(ChatState)

graph.add_node("chat_node", chat_node)

graph.add_edge(START, "chat_node")
graph.add_edge("chat_node", END)

chatbot = graph.compile()
```

Notice the compiled object is stored in a variable named **`chatbot`** rather than the generic `workflow` name used in earlier chapters — purely a naming convention that better reflects what this particular graph represents.

```
StateGraph(ChatState)
     ↓
add_node("chat_node", chat_node)
     ↓
add_edge(START, "chat_node")
add_edge("chat_node", END)
     ↓
compile()
     ↓
chatbot = <CompiledGraph>
```

A First Invocation

A single call confirms the flow works end-to-end: an initial state is built with one `HumanMessage`, and `chatbot.invoke()` runs it through `chat_node`.

```
initial_state = {
    'messages': [HumanMessage('What is the capital of India?')]
}

chatbot.invoke(initial_state)
# messages now holds:
#   HumanMessage("What is the capital of India?")
#   AIMessage("The capital of India is New Delhi.")
```

The final state's `messages` list contains both the human message and the new AI message. To extract just the answer text: `result['messages'][-1].content` — index `-1` gets the most recent (AI) message, and `.content` gets its text.

> ↻ **Quick Revision — Implementing the Chatbot**
>
> • chat_node reads state['messages'], calls model.invoke(messages) on the full list, and returns {'messages': [response]}.
>
> • Returning a single-item list lets the add_messages reducer append the new reply rather than overwrite history.
>
> • Graph assembly: StateGraph(ChatState) -> add_node -> add_edge(START->node->END) -> compile().
>
> • The compiled graph is conventionally stored as chatbot for this project.
>
> • result['messages'][-1].content extracts the latest AI reply's text from the final state.

3. Running the Chatbot & The Memory Problem

Turning It Into a Chat Loop

A single invocation is not a chatbot — a real chatbot needs an interactive loop that keeps accepting messages until the user chooses to leave. The pattern: loop forever, read user input, exit on a quit keyword, otherwise invoke the graph and print the AI's reply.

```
while True:

    user_message = input('Type here: ')
    print('User:', user_message)

    if user_message.strip().lower() in ['exit', 'quit', 'bye']:
        break

    response = chatbot.invoke({'messages': [HumanMessage(user_message)]})

    print('AI:', response['messages'][-1].content)
```

Run this and it does feel like a chatbot from the console: type a message, get a reply, keep going until `exit`/`quit`/`bye`. But it has a serious bug.

The Bug: No Memory Across Turns

Two demonstrations expose it:

1. Say *"Hi, my name is Nitesh"* → the bot replies with a normal greeting. Then ask *"Can you tell me my name?"* → the bot replies *"I don't have access to your personal information."* It was told the name one message earlier.
2. Say *"Add 10 to 100"* → correct answer, 110. Then say *"Now add 15 to the result"* → the bot returns a wrong, disconnected value (e.g. 25) instead of 125, because it no longer has "the result" from the previous turn in view.

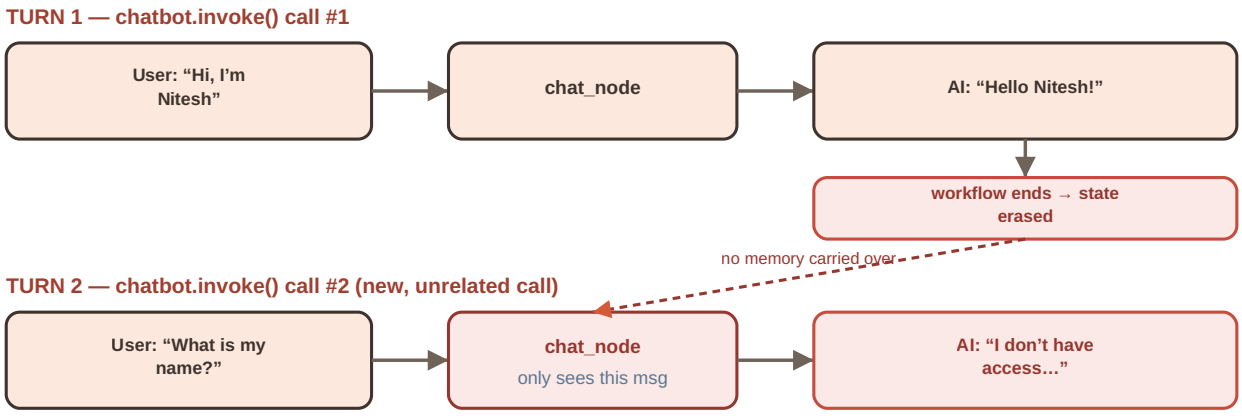

Figure 9.2 — Each while-loop iteration calls chatbot.invoke() independently. LangGraph state exists only for the duration of one invoke() run, so the second turn starts from a blank slate.

Root Cause

The full conversation history is *not* actually being sent to the LLM on every turn — even though every turn correctly appends its own message into a fresh list before invoking. The loop calls `chatbot.invoke(...)` once per turn, and **every call to `invoke()` starts the graph's state from scratch**. When execution reaches `END`, LangGraph's state for that run is gone from memory. The next loop iteration builds a brand-new `initial_state` containing only the newest `HumanMessage` — nothing from earlier turns survives, because those earlier turns belonged to a previous, already-finished invocation.

> **COMMON MISTAKE**
>
> It is tempting to think the fix is "send more messages into invoke() each time." The real problem is structural: state does not persist *between* separate invoke() calls at all, so there is nothing to accumulate unless something outside a single graph run stores it.

↻ Quick Revision — The Memory Problem

- The console chat loop calls chatbot.invoke() once per user turn.
- LangGraph state is scoped to a single invoke() run; it is discarded once execution reaches END.
- Each new loop iteration starts a fresh initial_state containing only the newest message.
- Symptom: the bot cannot recall earlier facts (a name) or earlier results (an arithmetic answer) from prior turns.
- The fix is not more messages per call — it is persisting state across separate invoke() calls.

4. Persistence in LangGraph

Definition

Persistence in LangGraph means that when a graph's execution reaches `END`, its state is **not** discarded — it is saved somewhere so a later `invoke()` can pick up exactly where the previous one left off, instead of starting from scratch.

Problem It Solves

This directly fixes Section 3's bug: without persistence, every `invoke()` is an island. With persistence, consecutive turns in the same conversation are linked, so the LLM keeps seeing the full accumulated history.

How It Works

There are two common places to store the saved state:

1. **A database.** State is written to persistent storage (e.g. SQL/NoSQL), so it survives even after the program itself is closed and restarted. This is the industry-standard choice for production chat systems, since a user may return to a conversation days later and still expect it to be remembered.
2. **In-memory (RAM).** State is kept in the running process's memory for as long as the program stays alive. This is simpler and is what this chapter's basic chatbot uses — but the state is lost the moment the program restarts.

LangGraph implements this through an object called a **checkpointer**. This chapter uses the in-memory implementation, **`MemorySaver`**, imported from `langgraph.checkpoint.memory`. A full, dedicated treatment of checkpointers — how they work internally, what else they enable (fault tolerance, human-in-the-loop, time travel) — is reserved for the next chapter; this chapter only wires one up to solve the immediate memory problem.

> **REAL WORLD INSIGHT**
>
> This is exactly why a WhatsApp-style AI assistant can be closed for four days and still remember what was discussed when reopened: production systems persist state to a database, not to RAM. A RAM-backed checkpointer only survives as long as the process is running.

↻ Quick Revision — Persistence

- Persistence means graph state survives past END instead of being discarded.
- Two storage options: database (survives program restarts; production-grade) or in-memory/RAM (simpler; lost on restart).
- LangGraph implements persistence via a checkpointer object.
- MemorySaver (from langgraph.checkpoint.memory) is the in-memory checkpointer used in this chapter.
- A full deep dive into checkpointers, threads, fault tolerance, and human-in-the-loop follows in the next chapter.

5. Implementing Persistence: Checkpointer, Threads & Config

Step 1 — Create a Checkpointer

A `MemorySaver` instance is created once, alongside the graph definition.

```
from langgraph.checkpoint.memory import MemorySaver

checkpointer = MemorySaver()
```

Step 2 — Register It at Compile Time

The checkpointer is attached to the graph through the `compile()` call, using the `checkpointer` parameter.

```
checkpointer = MemorySaver()

graph = StateGraph(ChatState)

graph.add_node("chat_node", chat_node)

graph.add_edge(START, "chat_node")
graph.add_edge("chat_node", END)

chatbot = graph.compile(checkpointer = checkpointer)
```

Step 3 — Identify Conversations with a Thread

A single running chatbot may be talking to many different users at once (or the same user across multiple separate conversations). LangGraph identifies each distinct conversation using a **`thread_id`**: one interaction between one user and the chatbot is one *thread*. Nitesh's conversation is one thread; Rahul's is a different thread; Amit's is another — each tracked and stored separately by the checkpointer.

Step 4 — Pass the Thread via config

Every call to `invoke()` now also passes a `config` dictionary identifying which thread this call belongs to, using the nested shape LangGraph expects: `{'configurable': {'thread_id': ...}}`.

```
thread_id = '1'

while True:

    user_message = input('Enter query: ')
    print('User : ', user_message)

    if user_message.strip().lower() in ["exit","quit","bye"]:
        break

    config = {'configurable':{'thread_id':thread_id}}

    response = chatbot.invoke({"messages":[HumanMessage(user_message)]}, config = config)

    print("AI :", response['messages'][-1].content)
```

Only two things changed from the earlier, memory-less loop: the graph was compiled with `checkpointer=checkpointer`, and `config=config` (carrying the `thread_id`) is now passed into every `invoke()` call. The node function, the state schema, and the edges are unchanged.

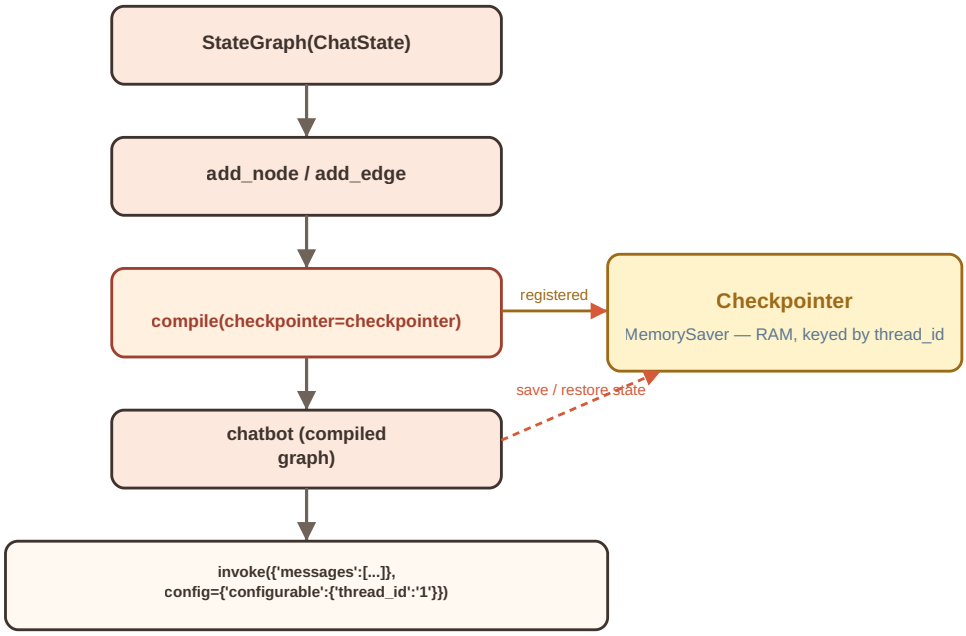

Figure 9.3 — The checkpointer is registered at compile time; every invoke() call supplies a thread_id through config, which the checkpointer uses to save and restore that conversation's state.

↻ Quick Revision — Checkpointer, Threads & Config

- checkpointer = MemorySaver() creates an in-memory checkpoint store.
- graph.compile(checkpointer=checkpointer) registers it on the compiled graph.
- A thread_id identifies one distinct conversation; different users/sessions get different thread_ids.
- config = {'configurable': {'thread_id': thread_id}} is passed into every invoke() call.
- No changes are needed to the state schema, node function, or edges to add persistence.

6. Running the Persistent Chatbot

Memory Now Works

Re-running the same two failing scenarios from Section 3, now with the checkpointer and thread_id in place:

- *"Hi, my name is Nitesh"* → greeting. *"What is my name?"* → **"Your name is Nitesh."**
- *"Can you add 10 to 100?"* → 110. *"Now can you multiply the result with 2?"* → **"110 × 2 = 220."**

Both previously-broken cases now work, because each new turn's `invoke()` call restores the same thread's saved state before running `chat_node`.

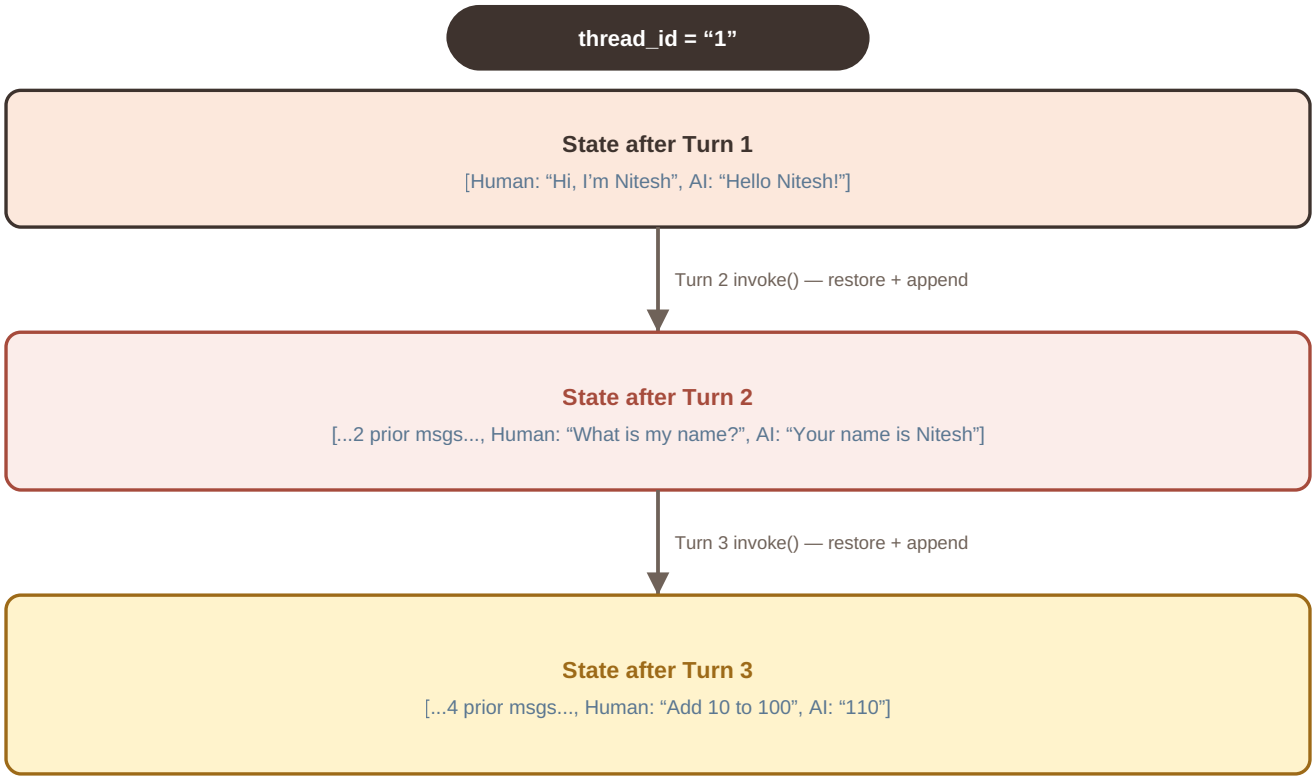

Figure 9.4 — On a fixed thread_id, each new turn restores the previously saved messages and add_messages appends the new exchange, so the accumulated history keeps growing turn over turn.

What Happens Behind the Scenes

1. Turn 1 runs; state ends up holding the human message and the AI reply. When the run reaches `END`, that state is saved into RAM, keyed by `thread_id`.
2. Turn 2 begins. Instead of starting from an empty state, LangGraph fetches the previously saved state for that same `thread_id` and uses it as the starting point.
3. The new turn's message is merged in via the `add_messages` reducer — appended onto, not replacing, the restored history.
4. At `END` again, the now-larger state (all messages so far) is saved back, overwriting the previous checkpoint for that thread.
5. This restore → append → save cycle repeats every turn, so the message list keeps growing for as long as the conversation continues on that thread.

Inspecting Saved State

The compiled graph's `get_state()` method returns everything currently checkpointed for a given thread — useful for debugging what the chatbot actually "remembers."

```
chatbot.get_state(config = config)
```

The same `config` dictionary (with the matching `thread_id`) must be supplied, since the checkpointer needs to know *which* conversation's state to return. The output includes the full `messages` list accumulated so far for that thread, along with LangGraph's internal metadata about that checkpoint.

> **INTERVIEW TIP**
>
> If asked "how does a LangGraph chatbot remember earlier turns," the precise answer is: it doesn't send the raw conversation from client-side memory — the *checkpointer* restores previously saved graph state keyed by `thread_id` at the start of each new invoke() call, and the state reducer (here, add_messages) merges the new turn onto it.

↻ Quick Revision — Running the Persistent Chatbot

- With a checkpointer + thread_id, each new invoke() restores that thread's saved state before running.
- add_messages appends the new turn onto the restored history rather than replacing it.
- State is re-saved at the end of every run, so the saved history grows turn by turn.
- chatbot.get_state(config=config) returns the full checkpointed state for a given thread_id.
- The same config (thread_id) used for invoke() must be reused to inspect or continue that same conversation.

7. Limitations & What's Next

In-Memory Persistence Does Not Survive a Restart

Because `MemorySaver` stores state in RAM, restarting the program (or the notebook kernel) clears everything the checkpointer was holding. Asking *"What is my name?"* immediately after a restart fails again — not because persistence stopped working, but because the RAM it depended on was wiped along with the process. This is the trade-off of choosing RAM over a database: simplicity now, no durability across restarts.

> **BEST PRACTICE**
>
> For any production chatbot, use a database-backed checkpointer (not `MemorySaver`) so conversations survive process restarts, deployments, and multi-day gaps between user sessions.

Where This Project Goes Next

This chapter deliberately stopped at a basic working memory feature using the simplest possible checkpointer. The next video in this series is fully dedicated to *persistence* in depth, covering:

- What checkpointers are internally and how they store/version state.
- Threads in more depth — multiple conversations, multiple checkpoints per thread.
- Fault tolerance — recovering a workflow after a mid-run failure.
- Human-in-the-loop (HITL) — pausing a graph to wait for human input or approval.

Later chapters in the same chatbot project will add RAG (document-grounded answers), tool calling, a UI, and LangSmith tracing on top of this same skeleton.

↻ Quick Revision — Limitations & What's Next

- MemorySaver keeps checkpoints in RAM only; a program/kernel restart erases all saved conversations.
- Database-backed checkpointers are the production choice for durability across restarts.
- The next chapter is a deep dive into checkpointers, threads, fault tolerance, and human-in-the-loop.
- Later chapters extend this same chatbot skeleton with RAG, tools, a UI, and LangSmith tracing.

K. Key Takeaways

- **Chatbot workflow:** a LangGraph chatbot is a single-node sequential workflow (START → chat_node → END) driven by an external loop, one invocation per turn.
- **State design:** ChatState holds one field, messages, typed as Annotated[list[BaseMessage], add_messages], so any mix of Human/AI/System/Tool messages can accumulate.

- **add_messages reducer:** a BaseMessage-optimized, LangGraph-native equivalent of operator.add, used to append rather than overwrite message history.
- **The stateless-invoke bug:** every chatbot.invoke() call starts a fresh graph state; nothing from a previous, already-finished invocation survives into the next one.
- **Persistence:** saving graph state past END (to RAM or a database) so a later invoke() can restore and continue it, instead of starting over.
- **Checkpointer:** the LangGraph object that implements persistence; MemorySaver is the simple in-memory implementation used here.
- **Thread & config:** a thread_id identifies one distinct conversation; it is passed via config={'configurable': {'thread_id': ...}} on every invoke() call.
- **get_state():** chatbot.get_state(config=config) inspects the full checkpointed state for a given thread.
- **Limitation:** RAM-backed persistence does not survive a process restart; production systems use database-backed checkpointers.

R. Revision Sheet

Chatbot Workflow

Single-node sequential graph: START → chat_node (LLM call) → END, wrapped in an external chat loop.

ChatState

TypedDict with one field: messages: Annotated[list[BaseMessage], add_messages].

BaseMessage

Common parent of HumanMessage, AIMessage, SystemMessage, ToolMessage — lets one list hold any message type.

add_messages

LangGraph's built-in reducer for message lists; appends new messages, optimized for BaseMessage objects.

The Stateless-Invoke Problem

Each invoke() runs an isolated graph execution; state is discarded at END, so separate calls cannot see each other's history.

Persistence

Saving state past END so it can be restored by a future invoke() — storage options are RAM or a database.

Checkpointer

LangGraph object implementing persistence; MemorySaver is the RAM-backed implementation from langgraph.checkpoint.memory.

Thread ID

Identifier for one distinct conversation; different threads have independently saved states.

config

Dict passed to invoke(): {'configurable': {'thread_id': ...}} — tells the checkpointer which conversation to restore/save.

get_state()

chatbot.get_state(config=config) returns the full checkpointed state (messages + metadata) for that thread_id.

I. Interview Questions

Q1. Why does a basic LangGraph chatbot need only one node, and what are the roles of START and END in that graph?

Q2. Why is a chatbot's messages state field typed as list[BaseMessage] rather than list[str]?

Q3. What problem does the add_messages reducer solve, and how does it differ from LangGraph's default state-update behavior?

Q4. A chatbot answers a question correctly but forgets it entirely on the very next turn even though the loop passes a new message each time. What is the most likely root cause?

Q5. Explain what happens to a LangGraph graph's state once execution reaches END, and why that matters for building a conversational chatbot.

Q6. What is a checkpointer in LangGraph, and what is the difference between an in-memory checkpointer and a database-backed one?

Q7. What is a thread_id used for, and what would happen if two different users' conversations were invoked with the same thread_id?

Q8. Walk through exactly what changes in the code (state schema, node, edges, invoke calls) when moving from a stateless chatbot to a persistent one.

Q9. What does chatbot.get_state(config=config) return, and why must the config passed to it match the config used during invoke()?

Q10. Why would a chatbot using MemorySaver lose all conversation history after a server restart, and how would you fix that for production?

F. Further Reading

- LangGraph official documentation — Persistence concepts (checkpointers, threads, state).
- LangGraph official documentation — Memory concepts (short-term vs. long-term memory in agents).
- LangChain documentation — Message types (HumanMessage, AIMessage, SystemMessage, ToolMessage).
- LangGraph API reference — MemorySaver and the BaseCheckpointSaver interface.